Transformers for Next-Token Prediction

In class we discussed self-attention, multi-head attention, and how to combine them to build a **decoder-only Transformer** for **next-token prediction**.

For this homework, you will start from the Transformer notebook you studied, from the course GitHub repository: **GPT-teaching.ipynb**. The notebook already trains the model. This time you will change its pieces, measure what happens, and look at what its attention heads learned.

Call `torch.manual_seed(42)` at the start of every run and train every model for 2000 steps (use eval_iters=50). Every part needs 2-4 sentences in your own words.

Part 1: (20 points) Create a separate **python** script (or a clean notebook) with all the pieces needed to train and test this LLM. Train the model with:

Config A:	**Config B:**
o `batch_size=16` o `seq_size=16` o `embed_dim=128` o `n_heads=4` o `n_layers=4`	o `batch_size=16` o `seq_size=32` o `embed_dim=256` o `n_heads=8` o `n_layers=8`

- For each configuration:
  - Report the number of model parameters.
  - Plot the "training and validation loss" vs "steps" (in 100-step intervals).
  - Report the ***time*** needed to train for 2000 steps, and the ***hardware*** you used.
  - Generate 1000 tokens. Print the ***text*** generated and the ***time spent*** to generate it.
- **Explain:** Config B has ~8x more parameters and twice the sequence length. Compare the training times and explain.

Part 2: (45 points) Using Config A, train three models that each change ***one*** thing compared to your baseline:

- **A1: No causal mask:** remove the look-ahead mask (every token sees the whole sequence).
- **A2: No positional embedding:** do not add the positional embedding to the token embedding.

- **A3: One head:** `n_heads=1` (one head of size 128 instead of 4 heads of size 32).

Plot the validation loss vs steps of all four models (baseline + 3) in one figure, give a table of their final training and validation losses, and print the first 300 generated characters of each. Keep the baseline and no-mask models for Part 3.

- **Explain (no mask):** Compare its losses with the baseline. Is it a better model? Look at its generated text. What did it learn, why does that fail at generation, and why did the validation loss not catch it?
- **Explain (no positional embedding):** How does its performance change?
- **Explain (one head):** Do both models (A and A3) have the same number of parameters. What is different between them, then? Did it matter here? Where do you expect it to matter?

Part 3: (35 points) Look inside the attention.

- Modify `SingleHeadAttention` so it stores its attention weights (after the softmax) from the last forward pass, e.g. add `self.last_weights = scores.detach()` right after the softmax line, before multiplying by the values.
- Feed the first 16 characters of the dataset (`text[:16]`) to your trained "**Config A"** baseline (use `model.eval()` and `torch.no_grad()`).
- Plot the 16x16 attention weights as heatmaps (`plt.imshow`), with the characters as labels: rows = the character asking (query), columns = the character looked at (key). Plot the 4 heads of the ***first*** block (one subplot per head).
    - **Hint:** the heads are in `model.blocks[i].mha.heads` (use .cpu() on a GPU).
    - **Hint:** outline the diagonal (cell (i, i)) in every heatmap, e.g. with `matplotlib.patches.Rectangle`: left of it = past, right of it = future.
- **Explain:** Which side of the diagonal is zero, and why? Pick one head and describe what it looks at (e.g. the previous character).
- Repeat the plot with your **no-mask** model from Part 2. **Explain:** do you see attention to the future, especially to the next character? How does this explain its low loss?

Upload your code as a Zip file (script or notebook) and a PDF report with the requested plots, tables, generated text and answers.